Testing AI-Generated Code — Participant Handout

Russ Poldrack, March 24, 2026

Learning Objectives

By the end of this workshop, you will:

1. Understand why tests are essential for AI-generated code
2. Understand the concept of test-driven development (TDD)
3. Understand the need for human examination of AI-generated tests

Why Cohen's *d*?

Cohen's *d* is an effect size estimator that every psychology researcher knows — but its implementation hides surprising complexity (see Lin & Aloe, 2020 for detailed overview). Multiple formula variants exist, and AI tools frequently choose one without telling you which. This makes it a perfect case study for understanding why testing AI-generated code matters.

Variant	SD Formula	When to Use
Cohen's *d* (pooled)	$s_p = \sqrt{\frac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_1+n_2-2}}$	Equal variances assumed
Glass's Δ	$s_{control}$	Control group as reference
Cohen's *d* (average)	$s_{avg} = \sqrt{\frac{s_1^2+s_2^2}{2}}$	Unequal n, simple estimate
Hedges' *g*	$g = d * a,$ $c(m) = \frac{\Gamma(\frac{m}{2})}{\sqrt{\frac{m}{2}}\Gamma(\frac{m-1}{2})},$ $m = n_1 + n_2 - 2$	Small sample bias correction (Hedges, 1981)

Part 1: Initial code generation

We'll begin with the following prompt:

```
Write a Python function to compute Cohen's d for two independent
groups.
```

The code it produces will look clean and correct. But consider:

- Which formula variant did it use?

- Did it use n or n−1 in the denominator (Bessel's correction)?
- Does it apply Hedges' correction for small samples?
- What happens with edge cases (equal means, zero variance, n = 1)?

Discussion Question: *How would you verify that this code is correct?* You can't just look at the output — a wrong formula will still produce a number. You need to know what the right answer should be.

Part 2: Test Cases and AI-Generated Tests

Step 1: Brainstorm Test Cases

Before writing any code or asking the AI for help, think about what a correct Cohen's *d* function should do. What test cases would you want to check? Think about normal inputs, edge cases, and error conditions.

Use this space to jot down your ideas:

Step 2: Examine AI-Generated Tests

Now we'll prompt for test generation using the following:

```
Write a comprehensive pytest test suite for a function called cohens_d
that computes Cohen's d for two independent groups. The function takes
two lists of numbers and returns a float. Include edge cases.
```

Use the tables below to check the AI's tests against your brainstormed list.

Coverage Checklist

Did the AI's test suite cover the cases you identified?

Test Case	Covered?	Notes / Issues
Basic case with known answer		
Equal means → d = 0		
Unequal sample sizes		
Hedges' *g* correction		
Zero variance in one group		
Single observation		
Empty input		
Other: _____		

Pathology Checklist

Look for these common problems in AI-generated tests:

Pathology	What It Looks Like	Found?
Circular expected values	*AI computed the expected value using the same logic it would use to write the function*	
Overly loose assertions	`assert result is not None` or `assert isinstance(result, float)` instead of a specific value	
Smoke tests only	*Tests that the function runs without crashing, but never check correctness*	
Redundant tests	*Multiple tests exercising the same code path with different data*	
Missing error tests	*No tests for bad input — doesn't check that errors are raised*	
Magic numbers	*Expected values with no explanation of how they were derived*	

The Closed Loop Problem: When AI generates both the code and the tests, it can create a self-consistent but incorrect system. The tests pass because they encode the same assumptions (or errors) as the code. There is no external ground truth. This is why human judgment, based on domain expertise and known ground-truth values, is essential.

Part 3: Test-Driven Development with AI

Now we flip the workflow. Instead of asking the AI to write code and hoping it's correct, we'll write tests first — encoding exactly what we want — and then use those tests as a specification for the AI to implement against.

The TDD-with-AI Workflow

1. Fix the AI's tests: replace circular values with hand-calculated ones, add missing cases
2. Ask the AI to write code that passes your corrected tests
3. Run the tests and iterate if they fail
4. Review the final code for correctness beyond what the tests cover

Key Takeaways

1. Tests are essential for AI-generated code AI code can be confidently and silently wrong. Tests grounded in domain knowledge are your safety net.

2. AI-generated tests need human scrutiny AI tests can form a closed loop with AI code — both sharing the same blind spots. Your domain expertise is the irreplaceable ingredient.

3. Write tests first, then let AI implement TDD with AI means your tests become a specification. The AI fills in the implementation; you define the requirements.

Startup instructions (for Mac or Linux):

- Install uv: https://docs.astral.sh/uv/getting-started/installation/
- Clone this repository:
    - `git clone https://github.com/poldrack/ai-coding-workshop`
- Synchronize the packages:
    - `uv sync`
- Activate the virtual environment:
    - `source .venv/bin/activate`
- Confirm that the testing environment works:
    - `pytest test_cohensd.py`
    - Should show that 1 test passed.

For an introduction to using pytest: https://github.com/poldrack/pytest_tutorial

Pytest basics

Basic assertion

```
assert result == expected
```

Approximate comparison (essential for floats)

```
assert result == pytest.approx(expected, abs=0.001)
```

Asserting that an error is raised

```
with pytest.raises(ValueError):

    function_that_should_fail()
```

Testing for a specific error message:

```
with pytest.raises(ValueError, match="need at least 2 values"):
    cohens_d([5], [1, 2, 3])
```